

Revisão geral

Sessenta e seis questões sobre tudo que passou: do primeiro `print` ao último teste automatizado. Sem consulta na primeira volta.

Nome: _____

Data: _____

Dez blocos, na ordem em que o conteúdo foi dado.

Responda a caneta, no próprio papel.

Só depois de fechar tudo é que o computador entra.

Como responder

Esta revisão não vale nota. Ela vale uma coisa mais chata de conseguir: saber, com honestidade, o que você aprendeu de verdade e o que você só viu passar na tela.

Por isso a ordem importa.

A REGRA DA PRIMEIRA VOLTA

Responda tudo **sem abrir o editor, sem abrir os PDFs e sem rodar nada**. Se você não souber, escreva "não sei" e siga. Um "não sei" honesto é informação; um chute copiado do material não é.

Na segunda volta, aí sim: abra o código, rode o que dá para rodar, confira as questões de saída na tela e corrija a caneta de outra cor. O que você tiver errado na primeira volta e acertado na segunda é exatamente a sua lista de estudo.

Os tipos de questão

TIPO	O QUE SE ESPERA
------	-----------------

Escrita	Duas ou três frases suas. Não precisa de vocabulário bonito, precisa de precisão. "Ele guarda o outro objeto dentro" vale mais que "é uma relação de agregação estrutural".
Marque	Uma alternativa só, ou o verdadeiro/falso item por item.
Na tela	Escreva exatamente o que o Python imprime, linha por linha. Se der erro, escreva qual erro e em que linha.
Código	Escreva o código à mão. Indentação conta. Não se preocupe com acento nem com caractere fora do lugar.

Os dez blocos

1	Python antes da POO	1-7
	Variáveis, condições, laços, funções, listas e dicionários.	
2	Classes e objetos	8-15
	class, __init__, self, atributo e método, instâncias separadas.	
3	Herança e polimorfismo	16-24
	super(), sobrescrita, "é um", e quando não herdar.	
4	Encapsulamento	25-31
	O underline, a @property, e as portas do estado.	
5	Composição	32-37
	"Tem um": itens, inventário e delegação.	
6	O jogo por dentro	38-45
	Fases, progressão, bônus temporários, habilidades e a IA do inimigo.	
7	Refatoração	46-51
	Mudar a forma sem mudar o comportamento.	
8	Testes automatizados	52-59
	assert, unittest, sorteio, print e a regra que se testa.	
9	Leitura de código	60-62
	Trechos do seu próprio jogo, para dizer o que aparece na tela.	
10	Escreva você	63-66
	Quatro trechos para escrever do zero, à mão.	

Onde conferir depois: o bloco 1 está em [dia-01-fundamentos-python/](#); o 2 em [dia-02-classes-e-objetos/](#); os blocos 3 a 5 em [teoria-poo/](#) e nas partes 1 a 3 de [pratica-poo/](#); o bloco 6 nas partes 4 a 9; o 7 em [Aula Refatoração/](#); e o 8 em [Hoje/](#).

Python antes da POO

O chão de tudo. Se alguma destas sete falhar, o problema não é orientação a objetos.

01 Traduza este trecho de Portugol para Python. CÓDIGO

```
leia(idade)
se (idade >= 18) entao
    escreva("pode votar")
senao
    escreva("ainda nao")
fimse
```

EM PYTHON

02 No laço `for i in range(1, 6):`, quais valores `i` assume? Por que o 6 não entra? ESCRITA

03 Qual a diferença entre `=` e `==`? Escreva uma linha de código usando cada um, e diga em voz alta o que cada linha faz. ESCRITA

04 O usuário digita `3`. O que aparece na tela? NA TELA

```
n = input("Quantos? ")
print(n * 2)
```

E se a segunda linha fosse `print(int(n) * 2)`? Explique a diferença em uma frase.

05 Você quer repetir um menu **até** o usuário digitar `0`, sem saber quantas vezes isso vai levar. Marque a estrutura correta. MARQUE

- (a) `for`, porque todo laço em Python é `for`
- (b) `while`, porque o número de repetições depende do que for digitado
- (c) `if`, porque existe uma condição envolvida
- (d) `for` com `range(1000)`, que é o bastante para qualquer pessoa

Justifique a sua escolha em uma frase:

06 O que aparece na tela? Explique a segunda linha da saída. NA TELA

```
def dobro(n):
    print(n * 2)

x = dobro(4)
print(x)
```

SAÍDA

O que mudaria se a função terminasse em `return n * 2` em vez de `print`?

- 07 Lista e dicionário. Para cada situação abaixo, escreva qual dos dois você usaria e por quê, em meia linha. ESCRITA

SITUAÇÃO

LISTA OU DICIONÁRIO?

As seis fases do jogo, na ordem em que acontecem

A tecla que o jogador digita e a ação que ela dispara

Os itens dentro da mochila

Classes e objetos

A ideia de que o programa pode ser feito de coisas, e não só de passos.

- 08 Explique a diferença entre **classe** e **objeto** usando um exemplo que **não** seja do RPG e não seja o exemplo do material. ESCRITA

- 09 O que é o `self`? Por que ele aparece como primeiro parâmetro na definição do método, mas você não passa nada no lugar dele quando chama `jogador.curar(20)`? ESCRITA

- 10 Quando o `__init__` roda? Escreva a linha de código que faz ele rodar para a classe `Guerreiro`.

ESCRITA

11 O que aparece na tela? NA TELA

```
class Cofre:
    def __init__(self, saldo):
        self.saldo = saldo

a = Cofre(10)
b = Cofre(10)
a.saldo = a.saldo + 5
print(a.saldo, b.saldo)
```

SAÍDA

.....

Em uma frase: por que b não mudou?

12 Para cada nome tirado do seu jogo, marque se é **atributo** ou **método**. MARQUE

NOME	ATRIBUTO OU MÉTODO?
nome	
curar	
_vida	
esta_vivo	
defendendo	
receber_dano	

Qual é a pergunta de uma frase só que separa os dois?

- 13 Este código roda sem erro nenhum, mas a última linha quebra com `AttributeError`. Ache o problema e diga como se conserta. ESCRITA

```
class Aluno:
    def __init__(self, nome):
        self.nome = nome

    def falar(self):
        print(self.nome, "presente")

a = Aluno("Rita")
a.falar()
```

- 14 Descreva, em passos, tudo que acontece quando o Python executa `jogador = Guerreiro("Ana")`. Vá até o ponto em que a mochila do Guerreiro já tem poção dentro. ESCRITA

- 15 A classe `Personagem` tem este método: ESCRITA

```
def __str__(self):
    return f"{self.nome} ({self._vida}/{self._vida_maxima})"
```

O que `print(jogador)` mostra com ele? E o que mostraria se você apagasse o método?

Herança e polimorfismo

A parte que dá para explicar errado e mesmo assim funcionar. Aqui é para explicar certo.

- 16 Para cada par, escreva **É UM** (herança) ou **TEM UM** (composição). MARQUE

PAR	É UM / TEM UM
Mago → Personagem	
Personagem → Inventário	
PoçãoDeVida → Item	
Fase → inimigo	
Dragão → Personagem	
Personagem → BonusTemporario	

- 17 O `__init__` do Guerreiro é este: ESCRITA

```
class Guerreiro(Personagem):
    def __init__(self, nome):
        super().__init__(nome, 120, 18, 8, 2)
```

O que essa chamada faz? E o que quebra se você apagar a linha do `super()` e não escrever mais nada no lugar?

18 O Guerreiro reescreve o `atacar` da mãe: **ESCRITA**

```
def atacar(self, alvo):
    dano = self.calcular_dano()
    alvo.receber_dano(dano)
    print(self.nome, "atacou sem falhar e causou", dano, "de dano")
```

Comparando com o `atacar` de `Personagem`: o que ele tirou, e que efeito isso tem no jogo?

19 O Mago defende assim: **ESCRITA**

```
def defender(self):
    super().defender()
    self.curar(CURA_DA_DEFESA_DO_MAGO)
    print(self.nome, "recuperou", CURA_DA_DEFESA_DO_MAGO, "de vida")
```

O que se perde se a primeira linha for apagada? Diga o efeito concreto no combate, não a teoria.

20 Dentro de `combate()`, no `main.py`, existe uma linha só para o turno do inimigo: **ESCRITA**

```
inimigo.agir(jogador)
```

Existem seis inimigos diferentes e o `main.py` não tem nenhum `if` perguntando qual é qual. Explique por quê. Como se chama isso?

21 Você cria um sétimo inimigo, `class Lobo(Personagem)`, e não escreve nenhum `agir` dentro dele. Marque o que acontece quando chega a vez do Lobo. **MARQUE**

- (a) Erro: `AttributeError: 'Lobo' object has no attribute 'agir'`
 - (b) Nada acontece, o turno dele passa em branco
 - (c) Roda o `agir` de `Personagem`, que ataca
 - (d) O Python escolhe o `agir` do inimigo mais parecido
-

22 Na parte 3 você fez a poção herdar de `Personagem` antes de aprender composição, e o resultado foi absurdo. Escreva **duas** coisas que a poção passava a ter e que não fazem sentido nenhum para uma poção. **ESCRITA**

Complete a regra: só herde quando a frase "_____ " for verdadeira.

23 O Dragão escreve `mensagem_de_recarga()` e `efeito_da_habilidade()`, mas **não** escreve `turnos_de_recarga()`. Quantos turnos dura a recarga dele, e por quê? **ESCRITA**

24 Escreva a classe `Bandido`, inimigo novo, com estas regras: `CÓDIGO`

- herda de `Personagem`;
- vida 50, ataque 14, defesa 3, nenhuma poção;
- enquanto tiver mais da metade da vida, ataca normalmente;
- abaixo da metade, foge: imprime `"O Bandido recua"` e defende.

CLASSES.PY

Encapsulamento

Quem manda no estado de um objeto é o próprio objeto. Sete questões sobre o que isso custa e o que isso compra.

- 25 A vida do personagem chama `_vida`, com underline na frente. O Python **proíbe** alguém de escrever `jogador._vida = 999`? ESCRITA

Responda sim ou não, e explique o que o underline significa de verdade.

- 26 Com este código na classe: NA TELA

```
@property
def vida(self):
    return self._vida
```

Diga o que acontece em cada linha abaixo: funciona, ou dá erro?

LINHA

O QUE ACONTECE

`print(jogador.vida)`

`jogador.vida = 500`

`jogador.curar(30)`

- 27 Liste os **três** métodos de `Personagem` que são as únicas portas por onde `_vida` muda depois do `__init__`. ESCRITA

28 O `curar` tem duas defesas: `ESCRITA`

```
def curar(self, quantidade):  
    if quantidade <= 0:  
        return  
    self._vida = min(self._vida + quantidade, self._vida_maxima)
```

Explique o que cada uma impede. Dê um exemplo concreto de chamada que seria absurda sem elas.

29 No `receber_dano` aparecem dois `max(0, ...)`: `ESCRITA`

```
dano_real = max(0, dano_real)  
self._vida = max(0, self._vida - dano_real)
```

O primeiro protege contra uma coisa, o segundo contra outra. Quais?

30 O `Inventario` guarda os itens em `self._itens` e oferece `adicionar`, `tirar`, `listar`, `quantidade` e `esta_vazio`. `ESCRITA`

Se o `main.py` passasse a mexer direto em `jogador.inventario._itens`, o programa ainda rodaria. Então qual é o problema?

- 31 Esta classe põe underline no atributo e ainda assim vaza o estado. Ache o vazamento e conserte.

CÓDIGO

```
class Turma:
    def __init__(self):
        self._alunos = []

    def matricular(self, nome):
        self._alunos.append(nome)

    def alunos(self):
        return self._alunos

t = Turma()
t.matricular("Rita")
t.alunos().clear()
print(len(t.alunos()))
```

O QUE IMPRIME, E COMO CONSERTAR

.....

.....

.....

Composição

A outra metade da orientação a objetos: a que não é herança.

- 32 O `Personagem` tem um `Inventario`. Em que linha exata do código esse inventário nasce, e quem é o dono dele? ESCRITA

Se dois personagens forem criados, existem quantos inventários? Por quê?

- 33 Todo item tem a mesma assinatura: `usar(self, dono, inimigo)`. ESCRITA
- Cite um item que usa só o `dono`, um que usa só o `inimigo`, e explique por que mesmo assim os dois recebem os dois parâmetros.

- 34 No `abrir_mochila`, o `main.py` termina com uma linha só: ESCRITA

```
item.usar(jogador, inimigo)
```

Você inventa a `PocaoDeVeneno`. O que ela precisa ter para funcionar aqui sem que uma única linha do `main.py` mude?

35 O `tirar` devolve `None` quando o número não existe: **ESCRITA**

```
def tirar(self, numero):
    indice = numero - 1
    if indice < 0 or indice >= len(self._itens):
        return None
    return self._itens.pop(indice)
```

Onde esse `None` é conferido? Que erro apareceria na tela se ninguém conferisse e o jogador digitasse `9` com dois itens na mochila?

36 Escreva a classe `PedraDeAfiar`, item novo: **CÓDIGO**

- herda de `Item`, nome "Pedra de afiar";
- ao ser usada, dá `+3` de ataque permanente ao dono e imprime uma mensagem dizendo isso.

ITENS.PY

37 No `mostrar_status`, o personagem escreve: **ESCRITA**

```
print("Mochila:", self.inventario.quantidade(), "item(ns)")
```

Ele poderia contar os itens sozinho. Por que é melhor perguntar para o inventário? Dê o nome dessa ideia.

O jogo por dentro

Fases, progressão, efeitos temporários, habilidades e a IA do inimigo — as partes 4 a 9.

- 38 A `BonusTemporario` tem o comentário: *"Guarda quanto e por quantos turnos. Não sabe a que atributo pertence."* ESCRITA

Explique o que essa segunda frase quer dizer, e o que se perderia se a classe soubesse que é do ataque.

- 39 O Orc entra em fúria: `aplicar_bonus_ataque_temporario(6, 3)`. A partir daí, cada `iniciar_turno` chama `passar_turno()`. NA TELA

```
def passar_turno(self):
    if self.turnos == 0:
        return False
    self.turnos -= 1
    return self.turnos == 0
```

Preencha a tabela a partir do momento em que a fúria começa.

TURNOS	DEPOIS	DEVOLVEU	ATAQUE VOLTA AO NORMAL?
1º			
2º			
3º			
4º			

40 O método que aplica o bônus começa desfazendo o bônus antigo: **ESCRITA**

```
def aplicar_bonus_ataque_temporario(self, quantidade, turnos):
    if self.bonus_ataque.ativo():
        self.ataque -= self.bonus_ataque.quantidade
    self.ataque += quantidade
    self.bonus_ataque.comecar(quantidade, turnos)
```

Que bug essas duas primeiras linhas impedem? Descreva o que aconteceria com um jogador que bebe dois elixires de fúria seguidos sem elas.

41 O Orc decide assim: **ESCRITA**

```
if not self.bonus_ataque.ativo() and self.vida < self.vida_maxima * VIDA_BAIXA_ORC:
    print("O Orc entra em fúria")
    self.aplicar_bonus_ataque_temporario(FORCA_DA_FURIA, DURACAO_DA_FURIA)
    return
```

Apague o `not self.bonus_ataque.ativo() and`. O que o Orc passa a fazer, turno após turno, com a vida baixa?

42 A `Fase` guarda quatro coisas. Escreva quais. **ESCRITA**

Por que uma classe `Fase` é melhor do que quatro listas paralelas (`nomes`, `descricoes`, `inimigos`, `premios`) percorridas pelo mesmo índice?

- 43 Escreva as **três** coisas que `recompensar(jogador, fase)` faz quando o jogador vence uma fase, e diga qual delas depende da fase que acabou de ser vencida. **ESCRITA**

- 44 O `usar_habilidade` da mãe é o roteiro, e deixa três buracos para as filhas preencherem.

ESCRITA

```
def usar_habilidade(self, alvo):
    if not self.habilidade_disponivel():
        print(self.mensagem_de_recarga() + ".", "Faltam",
              self._recarga_habilidade, "turno(s)")
        return
    self.efeito_da_habilidade(alvo)
    self.iniciar_recarga(self.turnos_de_recarga())
```

Quais são os três buracos, e qual é a parte que **nenhuma** filha precisa repetir?

- 45 O Guerreiro usa a habilidade no turno 1 e tenta usar de novo no turno 2. O que aparece na tela na segunda vez? **NA TELA**

SAÍDA DA SEGUNDA TENTATIVA

Em que turno ela volta a funcionar? (A recarga do golpe é 3.)

Refatoração

O dia em que o jogo mudou de forma e não mudou de comportamento.

- 46 Complete a definição e explique como ela foi conferida naquele dia, já que ainda não existiam testes. **ESCRITA**

Refatorar é mudar _____ sem mudar _____.

- 47 Este é um número mudo. Reescreva a linha dando nome a ele, e diga onde o nome deve ficar no arquivo. **CÓDIGO**

```
if self.vida < self.vida_maxima * 0.5:
```

REESCRITO

- 48 Antes, a linha do sorteio do dano estava copiada em dois arquivos. Depois virou um método só:

ESCRITA

```
def calcular_dano(self):  
    return random.randint(self.ataque - VARIACAO_DO_DANO,  
                           self.ataque + VARIACAO_DO_DANO)
```

Ele **devolve** o dano em vez de imprimir. Cite duas coisas que isso permite e que um `print` no lugar do `return` impediria.

- 49 O menu do turno era um `if/elif` de quatro braços e virou a tabela `ACOES`, no formato `"tecla": (rótulo, função)`. **ESCRITA**

Você quer acrescentar uma quinta ação, "Fugir". Quantos lugares do arquivo você precisa tocar agora? E antes, com o `if/elif`, quantos eram?

- 50 Antes da refatoração, as quatro habilidades começavam com a mesma guarda de recarga, copiada nas quatro. Para onde essa guarda subiu, e o que sobrou dentro de cada filha? **ESCRITA**

- 51 Alguém diz que fez "só uma refatoração" no `esta_vivo`. Compare os dois e diga se a regra do dia foi respeitada. **ESCRITA**

```
# antes
def esta_vivo(self):
    return self._vida > 0

# depois
def esta_vivo(self):
    return self._vida >= 0
```

Testes automatizados

O último dia: passar para o computador a conferência que era feita no olho.

52 O que acontece quando um `assert` é verdadeiro? E quando é falso? ESCRITA

Por que se diz que, em teste, **silêncio é aprovação**?

53 Um arquivo com doze `assert` soltos, em sequência, tem um defeito grave quando o terceiro falha. Qual? ESCRITA

Cite duas coisas que o `unittest` resolve nesse caso.

54 O que o `setUp` faz, e quando ele roda? ESCRITA

Por que não basta criar **um** personagem no topo do arquivo e usar o mesmo em todos os testes?

- 55 O `calcular_dano` sorteia. Escreva as três saídas para testar o que sorteia, na ordem de preferência que o material deu, e diga o que cada uma custa. **ESCRITA**

#	SAÍDA	CUSTO
1ª		
2ª		
3ª		

- 56 Quase tudo no jogo termina em `print`, e `print` não se afirma. Você quer testar que o ataque do Guerreiro machuca. Sobre o que você escreve o `assert`? **ESCRITA**

- 57 Dois testes para a mesma regra. Marque o que **prende o teste ao lado de dentro** da classe, e explique em uma frase. **MARQUE**

```
# teste A
jogador.aplicar_bonus_ataque_temporario(6, 3)
self.assertEqual(jogador.bonus_ataque.turnos, 3)

# teste B
ataque_antes = jogador.ataque
jogador.aplicar_bonus_ataque_temporario(6, 3)
self.assertEqual(jogador.ataque, ataque_antes + 6)
```

- (a) O teste A
- (b) O teste B
- (c) Os dois igualmente
- (d) Nenhum dos dois

- 58 Escreva um teste de `unittest` completo, com `setUp`, provando que **curar não passa da vida máxima**. CÓDIGO

TESTES.PY

- 59 A regra do dia foi: **o jogo não sabe que os testes existem**. A seta aponta num sentido só.

ESCRITA

Desenhe a seta entre `testes.py` e os outros quatro arquivos, e diga o que passa a acontecer se alguém escrever `import testes` dentro do `main.py`.

Leitura de código

Trechos do seu próprio jogo. Diga o que sai na tela e o que sobra no estado.

- 60 O Goblin tem 40 de vida máxima e está com 10. `VIDA_BAIXA_GOBLIN` é `0.3`. Chega a vez dele:

NA TELA

```
def agir(self, alvo):  
    if self.vida < self.vida_maxima * VIDA_BAIXA_GOBLIN:  
        self.defender()  
        return  
    self.atacar(alvo)
```

SAÍDA

E se ele estivesse com 12 de vida? Justifique com a conta.

- 61 Um Mago com 70 de vida máxima está com 65 e escolhe defender. A cura da defesa dele é 10.

NA TELA

SAÍDA, LINHA POR LINHA

Com quanta vida ele fica? Por quê?

- 62 O jogador usa a Bomba (30 de dano) contra um Troll que tem 6 de defesa e **está defendendo**.

ESCRITA

Escreva a sequência de chamadas, do menu até a vida do Troll mudar, e diga quanto de dano ele leva de fato.

```
def receber_dano(self, quantidade):
    dano_real = quantidade - self.defesa
    if self.defendendo:
        dano_real = dano_real // 2
    dano_real = max(0, dano_real)
    self._vida = max(0, self._vida - dano_real)
```

SEQUÊNCIA DE CHAMADAS E O DANO FINAL

Quatro trechos do zero, à mão. Indentação conta; acento e vírgula fora do lugar, não.

- saldo guardado de forma encapsulada, começando em zero;
- `depositar(valor)`, que ignora valor menor ou igual a zero;
- `sacar(valor)`, que só saca se houver saldo e devolve `True` ou `False`;
- uma `@property saldo` para leitura.

CONTA.PY

- 64 Escreva a classe `ContaPoupanca`, que é uma `ContaBancaria` e acrescenta `render(taxa)`, somando ao saldo a porcentagem indicada. Reproveite o que já existe: nada de repetir o depósito. `CÓDIGO`

CONTA.PY, CONTINUAÇÃO

- 65 Escreva três testes de `unittest` para a `ContaBancaria` da questão 63: um do depósito que funciona, um do depósito inválido, e um do saque sem saldo. `CÓDIGO`

TESTES_CONTA.PY

- 66 Refatore este trecho sem mudar o comportamento. Aponte, antes de escrever, quais são os dois problemas dele. **CÓDIGO**

```
def mostrar(p):
    if p.tipo == "guerreiro":
        print("Guerreiro:", p.nome)
        print("Vida:", p.vida, "/", p.vida * 1)
    elif p.tipo == "mago":
        print("Mago:", p.nome)
        print("Vida:", p.vida, "/", p.vida * 1)
    elif p.tipo == "ladino":
        print("Ladino:", p.nome)
        print("Vida:", p.vida, "/", p.vida * 1)
```

OS PROBLEMAS, E O CÓDIGO REFEITO

A sua própria correção

Agora abra tudo: o editor, os PDFs, o código. Corrija a caneta de outra cor, questão por questão, sem apagar o que você tinha escrito. O erro riscado é a parte que ensina.

A CONTA QUE IMPORTA

Não conte quantas você acertou. Conte quantas você **não sabia e agora sabe**. Essa é a única medida honesta de uma revisão.

Checklist do que a UC pedia

Marque só o que você conseguiria **explicar para outra pessoa**, sem olhar o material.

- ☐ Traduzir condição, laço e função do Portugol para o Python
- ☐ Dizer a diferença entre função que imprime e função que devolve
- ☐ Escrever uma classe com `__init__`, atributos e métodos
- ☐ Explicar o que é o `self`
- ☐ Criar dois objetos da mesma classe e dizer por que eles não se misturam
- ☐ Herdar de uma classe e chamar `super().__init__()`
- ☐ Sobrescrever um método e ainda assim aproveitar o da mãe
- ☐ Explicar polimorfismo mostrando uma linha de código que não pergunta o tipo
- ☐ Dizer quando **não** se deve usar herança
- ☐ Proteger um atributo e abrir uma `@property` de leitura
- ☐ Explicar por que um objeto guarda outro objeto dentro de si
- ☐ Ler o `agir` de um inimigo e prever o que ele vai fazer
- ☐ Dar nome a um número mudo e extrair um método repetido
- ☐ Escrever um `assert` que falha quando deve falhar
- ☐ Escrever uma classe de `unittest` com `setUp`
- ☐ Testar código que sorteia e código que imprime

Se sobrou tempo. Pegue as três questões que você mais errou e transforme cada uma num teste automatizado dentro do seu `testes.py`. O que você não soube responder no papel, o computador passa a responder por você para sempre.

Ficou com dúvida em alguma questão depois de conferir? Anote o número aqui e leve. Uma dúvida escrita vira pergunta; uma dúvida solta vira nada.
